

Line Follower mit Reinforcement Learning

Entwicklung und Training eines intelligenten Agenten mit Reinforcement Learning

Hochschule Bochum
Lehrveranstaltung: Künstliche Intelligenz

Teammitglieder: Majd Fares, Mohamad Aldabaa
Semester: 5

Inhaltsverzeichnis

Inhaltsverzeichnis	2
1. Kurzfassung	3
2. Projektidee und Zielsetzung	3
3. Verwendetes Framework	4
4. Beschreibung der Umgebung	4
5. Agent	4
6. Formulierung als Reinforcement-Learning-Problem.....	5
6.1 Observation Space	5
6.2 Action Space.....	5
6.3 Reward-Funktion	6
6.4 Episodenende.....	6
7. RL-Algorithmus.....	6
8. Training	7
9. Evaluation.....	8
9.1 Trainingsverlauf.....	8
9.2 Vergleich mit Baselines	9
9.3 Generalisierung auf unbekannte Strecken	9
10. Ergebnisse.....	10
11. Diskussion	10
11.1 Reward Hacking: Fallstudie und Behebung.....	10
11.2 Vergleich PPO und DQN.....	11
11.3 Grenzen und Ausblick	11
12. Fazit	11
13. Reproduzierbarkeit	12
13.1 Installationsanleitung.....	12
13.2 Projektstruktur	12
Anhang	13
A.1 Wichtige Codeausschnitte.....	13
A.2 Weitere Abbildungen.....	13

Hinweis: Rechtsklick auf das Inhaltsverzeichnis und „Felder aktualisieren“ wählen, um die Seitenzahlen zu erzeugen.

1. Kurzfassung

Im Rahmen dieses Projekts wurde eine zweidimensionale Simulationsumgebung entwickelt, in der ein Agent (ein vereinfachtes Auto) lernen soll, einer gewellten, ringförmigen Linie zu folgen. Der Agent nimmt seine Umgebung ausschließlich über fünf binäre Bodensensoren wahr und kann in jedem Zeitschritt zwischen drei diskreten Lenkaktionen wählen. Ziel einer Episode ist das Absolvieren von zwei vollständigen Runden. Die Umgebung wurde mit Python und Gymnasium als eigene Umgebung implementiert; das Training erfolgte mit den Algorithmen PPO und DQN aus der Bibliothek Stable-Baselines3.

Beide trainierten Agenten erreichen auf der Trainingsstrecke eine Erfolgsrate von 100 % – auch von zufälligen, im Training nicht gesehenen Startpositionen aus – und übertreffen die Random-Baseline deutlich. Auf unbekannten Streckenformen generalisiert PPO vollständig (100 %), während DQN auf der kurvenreichsten Teststrecke auf 88 % zurückfällt. Das wichtigste Ergebnis neben dem erfolgreichen Training ist ein dokumentierter Fall von Reward Hacking: Unter der ursprünglichen Reward-Funktion lernte der Agent, in Schlangenlinien zu fahren, um mehr Belohnung zu sammeln. Durch eine Umstellung auf eine fortschrittsbasierte Reward-Funktion konnte dieses Verhalten vollständig beseitigt werden.

2. Projektidee und Zielsetzung

Das Projekt greift eine klassische Aufgabe der mobilen Robotik auf: den Linienfolger. Ein physischer Linienfolger-Roboter erkennt mit Infrarotsensoren eine dunkle Linie auf hellem Untergrund und regelt seine Lenkung so, dass er der Linie folgt. Üblicherweise wird dieses Verhalten von Hand programmiert. Die Kernfrage dieses Projekts lautet: Kann ein Agent dieselbe Fähigkeit ausschließlich durch Reinforcement Learning erwerben, also allein aus Belohnungssignalen, ohne dass ihm die Bedeutung der Sensoren oder eine Lenkstrategie einprogrammiert wird?

Das Problem eignet sich aus mehreren Gründen gut für Reinforcement Learning: Es besitzt einen kompakten, klar definierten Beobachtungsraum (fünf binäre Sensoren, also 32 mögliche Beobachtungen), einen kleinen diskreten Aktionsraum (drei Lenkaktionen), unmittelbares Feedback in jedem Zeitschritt sowie ein eindeutiges Erfolgs- und Misserfolgskriterium. Gleichzeitig ist die optimale Strategie nicht trivial, da der Agent aus lokalen, stark begrenzten Sensorinformationen ein global sinnvolles Fahrverhalten ableiten muss.

Am Ende soll der Agent zwei vollständige Runden auf einer gewellten Rundstrecke fahren, ohne die Linie zu verlieren – und dies auch von beliebigen Startpositionen und idealerweise auf Streckenformen, die im Training nicht vorkamen.

Folgende Vereinfachungen wurden bewusst vorgenommen: Das Auto fährt mit konstanter Geschwindigkeit (keine Beschleunigung, keine Trägheit), die Lenkung erfolgt in drei diskreten Stufen, und die Sensoren liefern binäre Werte statt kontinuierlicher Reflexionswerte. Diese Vereinfachungen halten den Zustandsraum klein und das Training schnell, ohne den Kern des Problems – die Regelung anhand lokaler Sensorik – zu verändern.

3. Verwendetes Framework

Aspekt	Beschreibung
Framework / Engine	Python + Gymnasium (eigene Umgebung)
Programmiersprache	Python 3
RL-Bibliothek	Stable-Baselines3 (PPO, DQN)
Rendering / Simulation	Pygame (2D, vereinfachtes kinematisches Modell)
Betriebssystem	Windows 11 (Entwicklung und Training)
Weitere Pakete	NumPy, Matplotlib (Auswertung und Plots)

Tabelle 1: Technische Übersicht

Python + Gymnasium wurde gewählt, weil Gymnasium die De-facto-Standardschnittstelle für Reinforcement-Learning-Umgebungen definiert (reset/step, Observation Space, Action Space, terminated/truncated). Eine eigene Umgebung nach diesem Standard lässt sich ohne Anpassungen mit etablierten RL-Bibliotheken wie Stable-Baselines3 trainieren und mit dem offiziellen Umgebungs-Checker von Gymnasium validieren. Gegenüber Engines wie Unity oder Godot entfällt der Engine-Overhead vollständig; die gesamte Logik bleibt in wenigen hundert Zeilen Python nachvollziehbar, was der Reproduzierbarkeit zugutekommt.

4. Beschreibung der Umgebung

Die Umgebung ist eine zweidimensionale Draufsicht auf eine geschlossene Rundstrecke. Die Strecke wird als gewellter Kreis erzeugt: Ein Basisradius wird durch eine Sinusfunktion moduliert ($r(\theta) = R + A \cdot \sin(f \cdot \theta)$), wodurch eine organisch geschwungene Linie mit Links- und Rechtskurven entsteht. Die Trainingsstrecke verwendet $R = 200$, $A = 50$ und $f = 3$. Die Linie hat eine feste Breite von 20 Pixeln; eine weiße Markierung kennzeichnet die Start-/Ziellinie.

In der Umgebung existieren zwei Objekte: die Strecke (statisch) und der Agent (ein Auto, dargestellt als Dreieck). Es gelten folgende Regeln:

- Das Auto bewegt sich in jedem Zeitschritt mit konstanter Geschwindigkeit vorwärts.
- Die einzige Steuerungsmöglichkeit ist die Lenkung (links / geradeaus / rechts).
- Eine Episode ist erfolgreich, wenn das Auto zwei vollständige Runden absolviert.
- Eine Episode ist fehlgeschlagen, wenn alle fünf Sensoren die Linie verlieren.
- Als Sicherheitsgrenze wird eine Episode nach 2500 Schritten abgebrochen (truncated).

Der Rundenfortschritt wird über den Winkel gemessen, den das Auto um das Streckenzentrum zurücklegt; ein akkumulierter Winkel von 2π entspricht einer Runde. Optional kann die Umgebung Episoden an zufälligen Positionen der Strecke starten (random_start), was für die Evaluation genutzt wird.

5. Agent

Der Agent repräsentiert ein einfaches autonomes Fahrzeug, vergleichbar mit einem Linienfolger-Roboter. Sein Zustand besteht aus Position (x, y) und Ausrichtung (Winkel). Die Bewegung folgt einem bewusst vereinfachten kinematischen Modell: In jedem Zeitschritt wird zunächst der Lenkwinkel um einen festen Betrag ($\pm 0,08$ rad) geändert – oder beibehalten – und anschließend das Auto um eine konstante Distanz (4 Pixel) in Blickrichtung bewegt. Es gibt keine Trägheit, kein Rutschen und keine Geschwindigkeitsregelung.

Der Agent erhält ausschließlich die Werte seiner fünf Bodensensoren. Diese sind auf einer Querleiste 25 Pixel vor dem Fahrzeug angeordnet (Abstand 8 Pixel zueinander) und liefern jeweils den Wert 1, wenn sich der Sensorpunkt über der Linie befindet, sonst 0. Der Agent kennt weder seine Position noch die Streckenform – die Sensorleiste ist sein gesamtes „Sichtfeld“.

Die einzige Entscheidung, die der Agent in jedem Zeitschritt treffen muss, ist die Wahl einer der drei Lenkaktionen. Die Herausforderung besteht darin, aus dem Sensormuster abzuleiten, auf welcher Seite die Linie wegdriftet, und entsprechend gegenzulenken – eine Zuordnung, die dem Agenten an keiner Stelle vorgegeben wird.

6. Formulierung als Reinforcement-Learning-Problem

6.1 Observation Space

Der Beobachtungsraum ist MultiBinary(5): ein Vektor aus fünf binären Werten, z. B. [0, 1, 1, 1, 0]. Damit existieren insgesamt 32 mögliche Beobachtungen.

Beobachtung	Bedeutung
S1 (ganz links)	1 = Linie unter dem äußeren linken Sensor – starke Abweichung nach rechts
S2 (links)	1 = Linie driftet leicht nach links aus der Fahrzeugmitte
S3 (Mitte)	1 = Fahrzeug ist über der Linie zentriert
S4 (rechts)	1 = Linie driftet leicht nach rechts aus der Fahrzeugmitte
S5 (ganz rechts)	1 = Linie unter dem äußeren rechten Sensor – starke Abweichung nach links

Tabelle 2: Observation Space

Die Beobachtung ist bewusst minimal gehalten: Sie enthält weder Position, Geschwindigkeit noch Lenkwinkel. Dadurch ist die gelernte Policy zwangsläufig positionsunabhängig – eine Eigenschaft, die sich später in der Generalisierung auf unbekannte Strecken auszahlt (siehe Abschnitt 9).

6.2 Action Space

Der Aktionsraum ist Discrete(3):

Aktion	Bedeutung
0	nach links lenken (Lenkwinkel $-0,08$ rad)
1	geradeaus (Lenkwinkel unverändert)

Aktion	Bedeutung
2	nach rechts lenken (Lenkwinkel +0,08 rad)

Tabelle 3: Action Space

6.3 Reward-Funktion

Im Projektverlauf wurden zwei Reward-Funktionen entwickelt. Die erste Version (v1, „presence“) belohnte die Anwesenheit auf der Linie:

```
R_v1 = +1,0   wenn der mittlere Sensor die Linie sieht
        -0,1   wenn die Linie sichtbar, aber nicht zentriert ist
        -100   wenn alle Sensoren die Linie verlieren (Episodenende)
        +100   pro vollständig absolvierter Runde
```

Diese Funktion führte zu Reward Hacking (Details in Abschnitt 11): Da jeder Schritt auf der Linie +1 einbringt, ist ein längerer Weg lukrativer als ein kurzer. Die stochastische Trainings-Policy lernte, in Schlangenlinien zu fahren, und benötigte dadurch ca. 1415 statt ca. 705 Schritte für zwei Runden – bei fast verdoppeltem Reward (ca. 1610 statt ca. 905). Formal wurde die Belohnung maximiert, das gewünschte Verhalten (effizientes Linienfolgen) jedoch verfehlt.

Die endgültige Version (v2, „progress“) belohnt stattdessen den Streckenfortschritt:

```
R_v2 = +50 · Δθ (vorwärts zurückgelegter Winkel um das Streckenzentrum)
        -0,1   zusätzlich, wenn die Linie sichtbar, aber nicht zentriert ist
        -100   wenn alle Sensoren die Linie verlieren (Episodenende)
        +100   pro vollständig absolvierter Runde
```

Der Gesamtfortschritt für zwei Runden ist mathematisch fixiert (4π , also ca. 628 Reward-Punkte zuzüglich 200 Rundenbonus ≈ 828). Schlangenlinien erhöhen den Fortschritt nicht – sie kosten durch zusätzliche Zentrierungsstrafen sogar Reward. Rückwärtsfahren erzeugt negativen Fortschritt und ist damit ebenfalls unattraktiv. Das Reward Hacking ist durch diese Konstruktion prinzipiell ausgeschlossen, nicht nur abgeschwächt.

6.4 Episodenende

- Erfolg (terminated): zwei vollständige Runden absolviert.
- Misserfolg (terminated): alle fünf Sensoren verlieren die Linie (Strafe –100).
- Abbruch (truncated): 2500 Schritte erreicht, ohne dass eines der obigen Kriterien eintrat.

Die Unterscheidung zwischen terminated und truncated folgt der Gymnasium-Konvention: Nur echte Episodenenden (Erfolg/Misserfolg) beeinflussen die Wertschätzung zukünftiger Belohnungen; der Zeitabbruch ist eine externe Grenze.

7. RL-Algorithmus

Trainiert und verglichen wurden zwei Algorithmen aus Stable-Baselines3: PPO (Proximal Policy Optimization, policy-basiert) und DQN (Deep Q-Network, wertbasiert). PPO wurde als

Hauptalgorithmus gewählt, da es für seine Trainingsstabilität und robusten Standard-Hyperparameter bekannt ist. DQN wurde als klassischer Vertreter wertbasierter Verfahren für kleine diskrete Aktionsräume ergänzt, um die Algorithmuswahl empirisch statt nur verbal zu begründen.

Parameter	PPO	DQN
Policy	MlpPolicy	MlpPolicy
Lernrate	$3 \cdot 10^{-4}$ (Default)	$1 \cdot 10^{-4}$ (Default)
Discount-Faktor γ	0,99	0,99
Trainingsschritte	200.000	200.000
Batch Size	64	32
Exploration	stochastische Policy (ent_coef = 0)	ϵ -greedy, exploration_fraction = 0,3 (angepasst), ϵ_{final} = 0,05
Weitere Parameter	n_steps = 2048, clip_range = 0,2	buffer_size = 10^6 , learning_starts = 1000 (angepasst), target_update = 10.000
Seed	0	0

Tabelle 4: Trainingsparameter (Abweichungen von den SB3-Defaults sind markiert)

Bei PPO wurden ausschließlich Standardwerte verwendet. Bei DQN war eine Anpassung notwendig: Mit dem Default exploration_fraction = 0,1 endet die Explorationsphase bereits nach 10 % des Trainings – zu früh, um die hinteren Streckenabschnitte überhaupt zu erreichen. Die Verlängerung auf 0,3 behob dieses Problem.

8. Training

Beide Agenten wurden jeweils 200.000 Zeitschritte lang in der Umgebung mit fester Startposition und der v2-Reward-Funktion trainiert (train.py). Jede Episode wurde über einen Monitor-Wrapper protokolliert (Gesamt-Reward, Episodenlänge); aus diesen Logs wurden die Trainingsplots erzeugt. Das Training lief vollständig auf der CPU eines gewöhnlichen Desktop-PCs [AMD Ryzen 9, 5000er-Serie] und dauerte pro Lauf nur wenige Minuten – ein Vorteil der bewusst kleinen Umgebung.

Während des Projekts traten zwei nennenswerte Probleme auf, die jeweils Anpassungen erforderten:

- Reward Hacking unter der v1-Reward-Funktion: Der Agent maximierte die Belohnung durch Schlangenlinienfahren statt durch effizientes Linienfolgen. Behoben durch die Umstellung auf die fortschrittsbasierte v2-Reward-Funktion (siehe Abschnitte 6.3 und 11).
- Unzureichende Exploration bei DQN: Mit den Default-Einstellungen erreichte DQN kein stabiles Lernen, da die ϵ -greedy-Exploration zu früh endete. Behoben durch exploration_fraction = 0,3.

Der Trainingsverlauf beider Algorithmen unterscheidet sich deutlich: PPO zeigt nach ca. 340 Episoden einen scharfen Durchbruch und hält anschließend ein stabiles Plateau nahe dem Maximalreward. DQN benötigt etwa dreimal so lange bis zum Durchbruch (ca. Episode 1100) und

oszilliert auch danach sichtbar – ein bekanntes Stabilitätsmerkmal wertbasierter Verfahren, das sich hier direkt beobachten lässt.

9. Evaluation

9.1 Trainingsverlauf

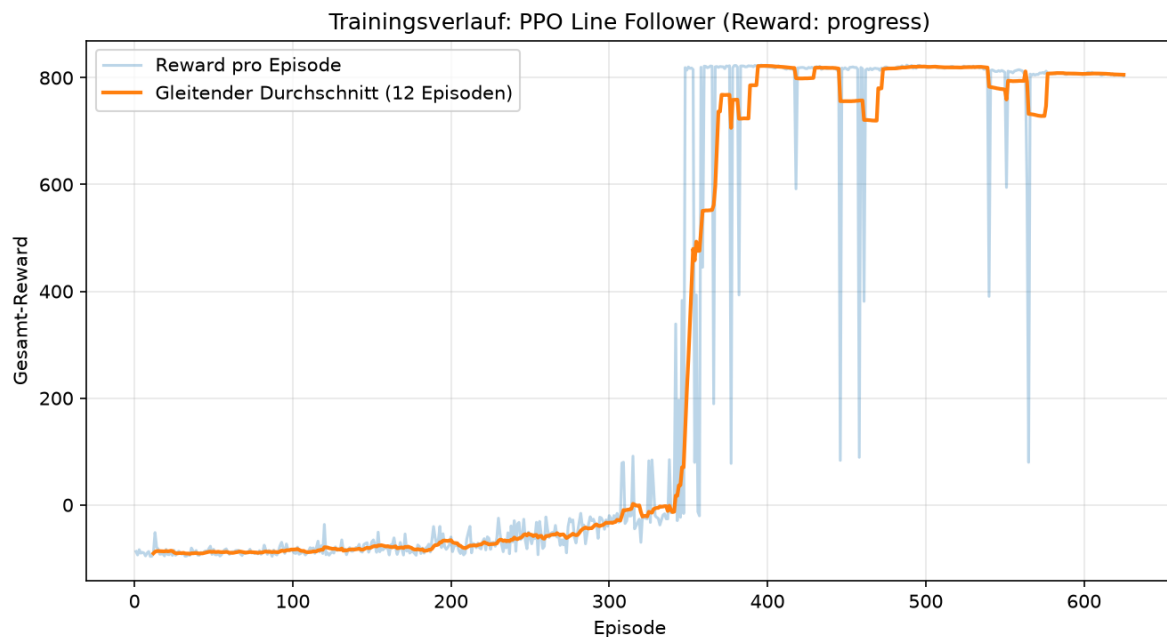


Abbildung 1: Trainingsverlauf PPO (Reward pro Episode, gleitender Durchschnitt)

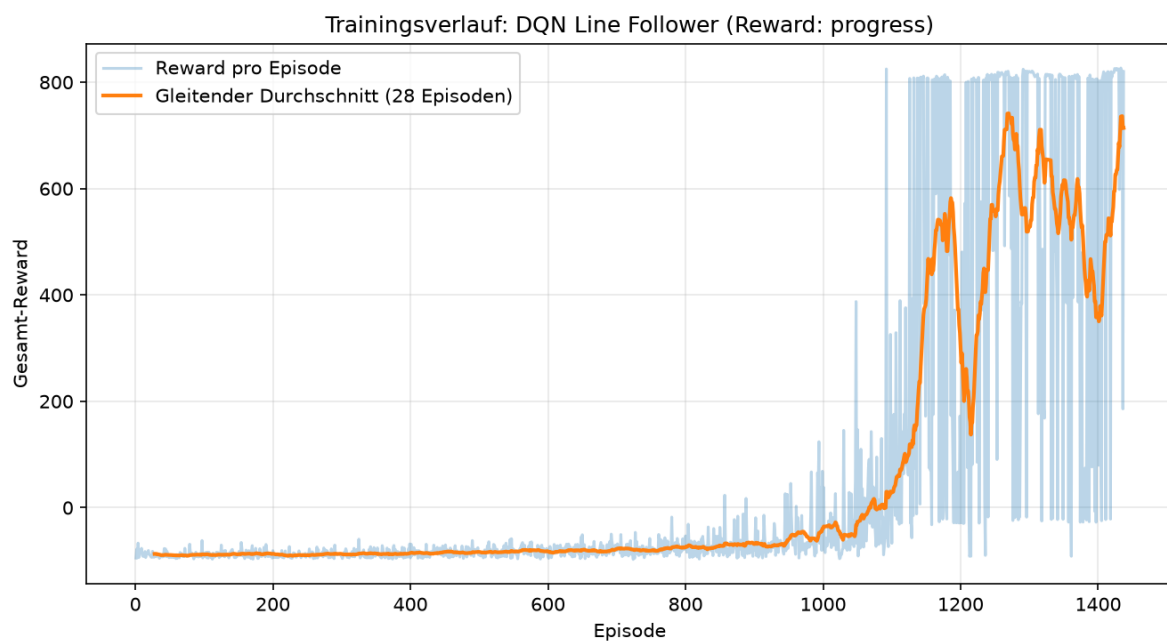


Abbildung 2: Trainingsverlauf DQN (Reward pro Episode, gleitender Durchschnitt)

Beide Kurven zeigen den klassischen Verlauf eines erfolgreichen RL-Trainings: eine lange Phase niedriger Rewards (der Agent verliert die Linie nach wenigen Schritten, Reward ≈ -80 bis -100), einen vergleichsweise abrupten Durchbruch und anschließend ein Plateau nahe dem theoretischen Maximum von ca. 828. Der Vergleich der beiden Plots belegt die höhere Trainingsstabilität von PPO.

9.2 Vergleich mit Baselines

Alle Verfahren wurden unter identischen Bedingungen evaluiert: 50 Episoden mit zufälligen, per Seed reproduzierbaren Startpositionen, wobei jede Policy exakt dieselbe Sequenz von Startpositionen erhielt (evaluate.py). Als Baselines dienen eine Random Policy (zufällige Lenkaktionen) und eine regelbasierte Policy, die auf den gewichteten Schwerpunkt der aktiven Sensoren zulenkt.

Verfahren	Ø Reward	± Std	Ø Schritte	Erfolgsrate	Ø Runden
Random Policy	-80,3	13,8	23,2	0 %	0,00
Regelbasierte Policy	828,7	0,4	705,3	100 %	2,00
PPO (trainiert)	803,2	0,3	669,9	100 %	2,00
DQN (trainiert)	824,5	0,3	681,0	100 %	2,00

Tabelle 5: Vergleich der trainierten Agenten mit den Baselines (Trainingsstrecke, 50 Episoden, zufällige Starts, Seed 0)

Beide trainierten Agenten erreichen die maximale Erfolgsrate und benötigen dabei weniger Schritte als die regelbasierte Policy (PPO: 669,9; DQN: 681,0; Regel: 705,3) – sie haben also eine effizientere Fahrlinie gefunden als die von Hand entworfene Strategie. Der etwas niedrigere Reward von PPO (803,2) gegenüber DQN (824,5) erklärt sich durch unterschiedliche Fahrstile: PPO schneidet Kurven und nimmt dafür Zentrierungsstrafen in Kauf, DQN fährt sauber zentriert. Interessant ist, dass beide Agenten von zufälligen Startpositionen aus zu 100 % erfolgreich sind, obwohl sie ausschließlich mit fester Startposition trainiert wurden – eine direkte Folge der positionsunabhängigen Beobachtung.

9.3 Generalisierung auf unbekannte Strecken

Zusätzlich wurden beide Agenten auf zwei Streckenformen evaluiert, die im Training nie vorkamen: eine sanftere Strecke ($A = 30$, $f = 2$) und eine kurvenreichere Strecke ($A = 45$, $f = 5$).

Strecke	Verfahren	Ø Reward	± Std	Ø Schritte	Erfolgsrate
sanfter (A=30, f=2)	Regelbasiert	828,9	0,1	642,0	100 %
	PPO	806,4	0,3	608,0	100 %
	DQN	828,9	0,2	617,3	100 %
welliger (A=45, f=5)	Regelbasiert	828,7	0,2	805,2	100 %
	PPO	797,7	0,3	777,9	100 %
	DQN	706,4	293,7	694,7	88 %

Tabelle 6: Generalisierung auf unbekannte Streckenformen (je 50 Episoden, zufällige Starts)

PPO generalisiert vollständig: 100 % Erfolg auf beiden unbekannten Strecken. DQN besteht die sanfte Strecke fehlerfrei, verliert auf der kurvenreichsten Strecke jedoch in etwa jeder achten Episode die Linie (88 %, hohe Standardabweichung von 293,7). Der sorgfältig zentrierte Fahrstil von DQN erweist sich auf unbekannten scharfen Kurven als weniger robust als der kurvenschneidende Stil von PPO.

10. Ergebnisse

Der Agent hat das gewünschte Verhalten vollständig gelernt. Der Lernerfolg zeigt sich an drei Stellen: erstens am Trainingsverlauf (vom Niveau der Random-Baseline auf das Plateau nahe dem Maximalreward), zweitens an der Erfolgsrate von 100 % über 50 Episoden mit zufälligen Startpositionen und drittens am direkten Vergleich mit den Baselines – beide Agenten schlagen die Random Policy um Größenordnungen und fahren effizienter (weniger Schritte pro Doppelrunde) als die regelbasierte Policy.

Die Agenten sind auf der Trainingsstrecke vollständig zuverlässig (Standardabweichung des Rewards < 0,5 über 50 Läufe). Schwächen zeigen sich nur außerhalb der Trainingsverteilung: DQN verliert auf einer deutlich kurvenreicheren, unbekannten Strecke in 12 % der Episoden die Linie. PPO zeigt diese Schwäche nicht, erkauft sich seine Robustheit jedoch mit einem leicht unsauberen, kurvenschneidenden Fahrstil (niedrigerer Reward bei höherer Effizienz).

Gegenüber den Baselines ergibt sich damit folgendes Bild: Beide RL-Agenten erreichen die Erfolgsrate der handkodierten Regel und übertreffen deren Effizienz – der eigentliche Erkenntnisgewinn liegt jedoch darin, dass diese Leistung ausschließlich aus Belohnungssignalen gelernt wurde, ohne dass die Sensor-zu-Lenkung-Zuordnung an irgendeiner Stelle vorgegeben war.

11. Diskussion

11.1 Reward Hacking: Fallstudie und Behebung

Das lehrreichste Ergebnis des Projekts war ein realer Fall von Reward Hacking. Unter der v1-Reward-Funktion (+1 pro zentriertem Schritt) zeigte der Trainingsmonitor Plateau-Episoden mit ca. 1610–1620 Reward bei ca. 1415 Schritten – für eine Strecke, die deterministisch in ca. 705 Schritten zu bewältigen ist. Die Erklärung: Da jeder Schritt auf der Linie +1 einbringt, verdoppelt ein doppelt so langer Weg die Belohnung. Die stochastische Trainings-Policy lernte daher Schlangenlinien zu fahren. Formal wurde die Belohnung maximiert; das intendierte Verhalten wurde verfehlt – exakt das in der Aufgabenstellung beschriebene Reward-Hacking-Phänomen, hier mit eigenen Messdaten belegt (results/v1_presence_monitor.csv).

Die Behebung erfolgte nicht durch eine Strafe (eine kleine Zeitstrafe hätte den Anreiz nur abgeschwächt, solange der Netto-Reward pro Schritt positiv bleibt), sondern durch eine strukturelle Änderung: Die v2-Reward-Funktion belohnt den Fortschritt entlang der Strecke, dessen Gesamtsumme für zwei Runden fixiert ist. Unter v2 fahren beide Agenten die Doppelrunde in weniger Schritten als die regelbasierte Policy – derselbe Algorithmus, der zuvor das

Belohnungssystem ausnutzte, fährt nun effizienter als die Referenzlösung. Die zentrale Erkenntnis: Die Reward-Funktion definiert das Verhalten stärker als der Algorithmus.

11.2 Vergleich PPO und DQN

Beide Algorithmen lösen die Aufgabe, unterscheiden sich aber charakteristisch. PPO trainiert schneller (Durchbruch nach ca. 340 statt ca. 1100 Episoden) und stabiler (glattes Plateau statt anhaltender Oszillation) und benötigte keinerlei Hyperparameter-Anpassung. DQN erforderte eine verlängerte Explorationsphase und blieb im Training sichtbar unruhiger. Bemerkenswert ist, dass beide Verfahren aus identischen Belohnungen unterschiedliche Fahrstile entwickelten: PPO schneidet Kurven (vermutlich begünstigt durch den Discount-Faktor, der früher erhaltene Belohnung höher gewichtet und damit schnellere Runden implizit belohnt), DQN fährt sauber zentriert. In der Generalisierung erwies sich der PPO-Stil als robuster, der DQN-Stil auf der Trainingsverteilung als präziser – ein klassischer Robustheit-Präzision-Kompromiss.

11.3 Grenzen und Ausblick

Die Lösung hat klare Grenzen: Das kinematische Modell ist stark vereinfacht (konstante Geschwindigkeit, keine Trägheit), die binären Sensoren machen den Agenten auf sehr scharfen Kurven anfällig dafür, die Linie zwischen zwei Zeitschritten vollständig zu verlieren, und die winkelbasierte Rundenzählung setzt ringförmige Strecken voraus. Mit mehr Zeit wären folgende Erweiterungen interessant: kontinuierliche Sensorwerte (Abstand zur Linienmitte statt binär), eine variable Geschwindigkeit als zusätzliche Aktionsdimension, Training auf zufällig generierten Strecken (Domain Randomization) zur weiteren Verbesserung der Generalisierung sowie ein Hyperparameter-Tuning für DQN, um dessen Robustheitslücke zu schließen.

12. Fazit

Entwickelt wurde eine vollständige Reinforcement-Learning-Pipeline für einen Linienfolger: eine eigene Gymnasium-Umgebung mit klar definiertem Beobachtungs- und Aktionsraum, zwei trainierte Agenten (PPO und DQN), zwei Baselines sowie eine quantitative Evaluation einschließlich Generalisierungstests auf unbekannten Strecken. Beide Agenten lernen das Zielverhalten zuverlässig und übertreffen die Effizienz der handkodierten Referenzlösung.

Die wichtigste Erkenntnis des Projekts betrifft das Reward Design: Die erste, intuitiv plausible Reward-Funktion wurde vom Agenten ausgenutzt (Reward Hacking durch Schlangenlinienfahren) und musste strukturell – nicht nur graduell – korrigiert werden. Erst die fortschrittsbasierte Belohnung erzeugte das intendierte Verhalten. Daneben zeigte der empirische Algorithmusvergleich, dass PPO und DQN aus identischen Belohnungen unterschiedliche Fahrstile mit unterschiedlichen Robustheitseigenschaften entwickeln.

Über Reinforcement Learning wurde damit vor allem gelernt: Der Agent optimiert exakt das, was belohnt wird – nicht das, was gemeint ist. Sorgfältiges Reward Design, quantitative Evaluation gegen

Baselines und Tests außerhalb der Trainingsverteilung sind keine Formalitäten, sondern der Kern der Methode.

13. Reproduzierbarkeit

13.1 Installationsanleitung

```
# Projektordner entpacken, dann im Ordner:
python -m venv .venv
.venv\Scripts\activate          # Windows
source .venv/bin/activate       # Linux / macOS
pip install -r requirements.txt

python train.py                 # Training (ALGO in train.py: "PPO" oder "DQN")
python evaluate.py              # Baseline-Vergleich + Generalisierung
python watch.py                 # Demo des trainierten Agenten (watch.py dqn für DQN)
python manual_drive.py          # Umgebung manuell testen (Pfeiltasten)
```

Training und Evaluation verwenden feste Seeds (Seed 0) und sind damit exakt reproduzierbar. Die Umgebung wurde mit dem offiziellen Gymnasium-Umgebungs-Checker (`gymnasium.utils.env_checker.check_env`) validiert.

13.2 Projektstruktur

```
LineFollower/
|-- README.md
|-- requirements.txt
|-- track.py                  # Streckengeometrie + Roboter-Kinematik + Sensorik
|-- line_follower_env.py      # Gymnasium-Umgebung (Reward, Episodenlogik)
|-- train.py                  # Training (PPO/DQN) + Trainingsplot
|-- evaluate.py               # Baseline-Vergleich + Generalisierungstest
|-- watch.py                  # Demo des trainierten Agenten
|-- manual_drive.py           # manueller Test der Umgebung
|-- models/                   # trainierte Modelle (.zip), inkl. v1-Modell
|-- results/                  # Trainingsplots + Episoden-Logs (v1 und v2)
|-- docs/
    |-- dokumentation.pdf
```

Anhang

A.1 Wichtige Codeausschnitte

Kern der v2-Reward-Funktion und Episodenlogik (line_follower_env.py, gekürzt):

```
progress = self._forward_progress() # vorzeichenbehafteter Winkel pro Schritt
if obs.sum() == 0:                  # Linie komplett verloren
    reward = -100.0
    terminated = True
elif self.reward_mode == "progress":
    reward = 50.0 * progress         # Belohnung fuer Vorwaertsfortschritt
    if obs[2] != 1:
        reward -= 0.1               # nicht zentriert

self._angle_travelled += progress    # Rundenzaehlung ueber Winkel
if self._angle_travelled >= 2 * math.pi:
    self._angle_travelled -= 2 * math.pi
    self.laps += 1
    reward += 100.0
    if self.laps >= 2:
        terminated = True           # Erfolg: 2 Runden
```

A.2 Weitere Abbildungen

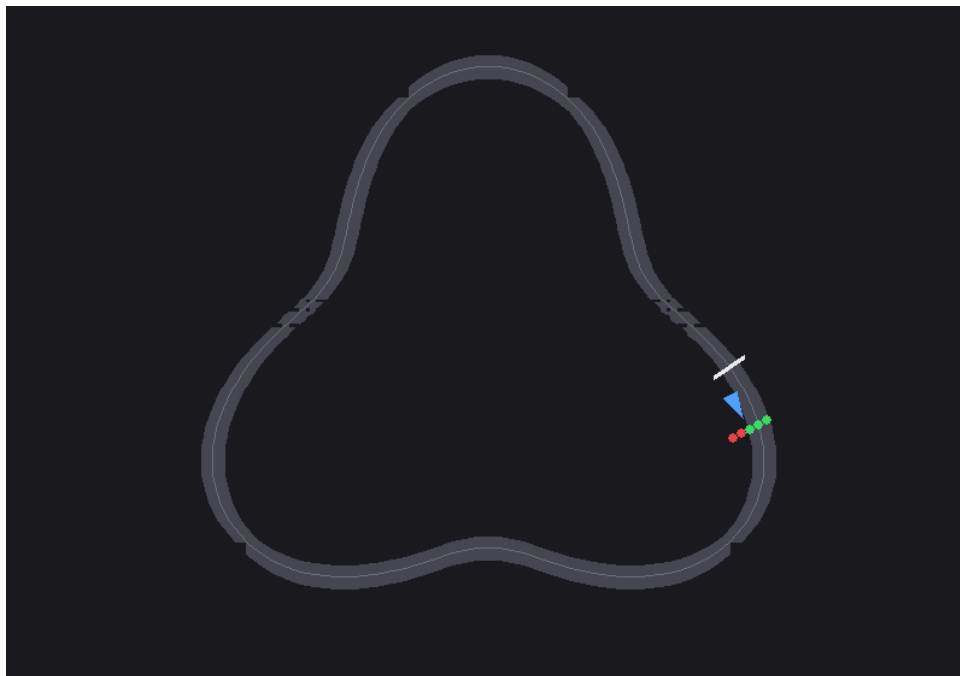


Abbildung 3: Die Simulationsumgebung mit Strecke, Agent (blau), Sensoren (grün/rot) und Start-/Ziellinie (weiß)

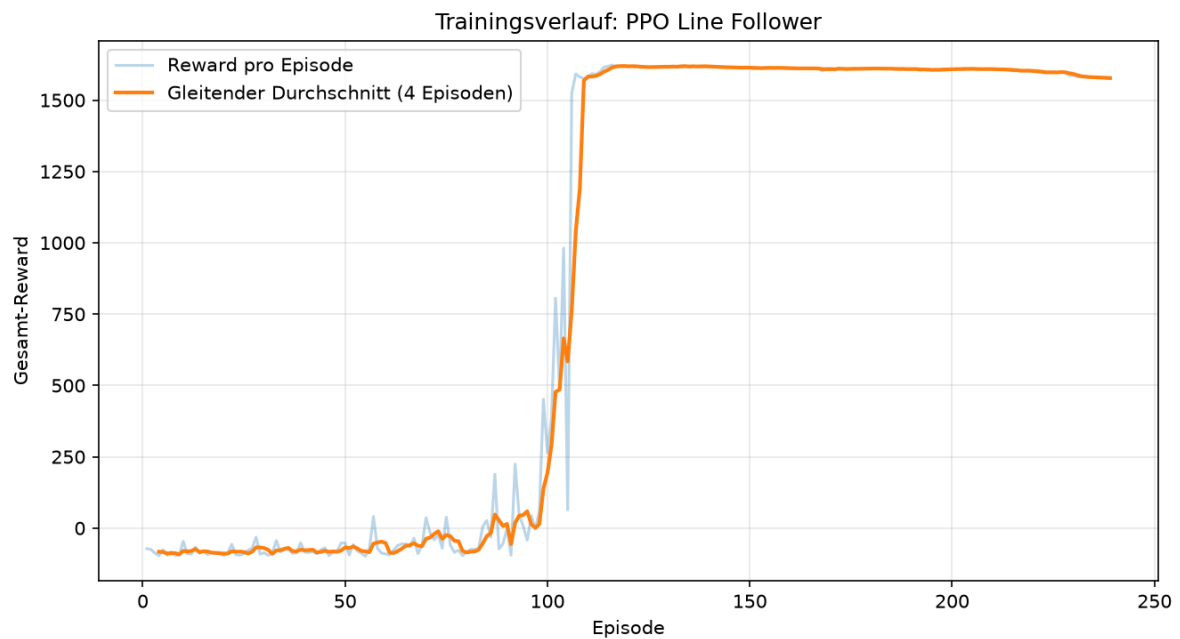


Abbildung 4: Trainingsverlauf unter der v1-Reward-Funktion – Plateau bei ca. 1610 statt der für eine effiziente Doppelrunde erwartbaren ca. 905 (Reward Hacking durch Schlangenlinienfahren)